Informe Técnico Integrador · Segunda Entrega Parcial (Unidades 4 a 8)

Proyecto «Food Store» · Base de Datos II · UTN · Ciclo 2026
Equipo: Morón / Menéndez · Organización Git: **main** (U4–U6), **NoSQL** (U7), **sqlite** (U8 + Integración U4–U8)

1. Elementos Implementados por Unidad (Unidades 4 a 8)

Unidad	Objetivos Acreditados	Artefactos (Specs, Scripts y Docs)
U4 · Normalización avanzada y redundancia	Refactorización de *control_lote_almacen* a **FNBC** validada contra DF reales; migración sin pérdida en transacción reversible con **plan de reversión obligatorio**; desnormalización controlada del Top-5 categorías del día con 5 triggers y consulta de auditoría.	• **Specs Kiro:** .kiro/specs/fnbc-y-desnormalizacion/ • **SQL:** db/preparar_food_store.sql, db/tp_fnbc_control_lote.sql, db/tp_desnormalizacion_top_categorias.sql • **Doc:** docs/informe_unidad4_fnbc.md
U5 · Administración y seguridad	Esquema de usuarios, roles de grupo (NOLOGIN) y login con **mínimo privilegio** y permisos de columna; auditoría nativa activada y probada con **simulacro de acceso indebido** y propuesta de contención; auditoría de permisos asistida por IA y anonimización determinista.	• **Specs Kiro:** .kiro/specs/roles-food-store/ • **SQL:** sql/roles.sql, sql/auditoria_test.sql, sql/simulacro_incidente.sql, sql/usuario_anon.sql • **Docs:** docs/informe_auditoria.md, docs/informe_incidente.md, docs/verificacion_roles.txt, docs/verificacion_equivalencia.md
U6 · Mantenimiento	Plan de mantenimiento automatizado (respaldo custom *pg_dump -Fc*, estadísticas, *VACUUM ANALYZE*, *health_check*); **simulacro de desastre contra reloj** con medición de RTO/RPO e informe post-incidente.	• **Specs:** SPEC_U6.md, .kiro/specs/mantenimiento-u6/ • **Scripts:** db/backup_foodstore.ps1, db/restore_test.ps1, db/maintenance.sql, db/health_check.sql • **Doc:** docs/informe_u6.md
U7 · NoSQL	Modelado documental de pedidos y detalles en **MongoDB** con documentos embebidos y *NumberDecimal* sobre pedidos reales (IDs 1 a 5); comparación práctica y justificación frente a PostgreSQL.	• **Scripts:** sql/consultas_mongodb.js, sql/consultas_postgresql.sql • **Doc:** docs/informe_nosql.md
U8 · Bases móviles e integración	Base **SQLite (STRICT)** offline para feria, sincronización y semáforo especificados en Kiro y generados con OpenCode; casos de conflicto **C1 a C7**; **code review final asistido por IA** de todos los scripts de la cursada con decisión humana documentada.	• **Specs Kiro:** .kiro/specs/movil-sqlite-sync/, docs/kiro/ • **Scripts/DB:** movil/schema_sqlite.sql, cargar_cache.sql, semaforo.sql, pruebas_sqlite.sql, simulador.py, foodstore_movil.db, movil/tests/ (25 tests) • **Docs:** docs/informe_tp8.md, docs/evidencia_*.md

2. Metodología de Pruebas y Resultados de Unidades 4 a 7

• **Unidad 4 (Transacciones reversibles, plan de reversión y EXPLAIN ANALYZE):** En *db/tp_fnbc_control_lote.sql* la migración hacia *control_lote_responsable* (R1) y *responsable_deposito* (R2) corre dentro de *BEGIN ... COMMIT* validando con un bloque *DO $$* que *(original EXCEPT vista) UNION ALL (vista EXCEPT original)* arroje **0 filas**; al final se prueba el **plan de reversión obligatorio** dentro de *BEGIN ... ROLLBACK* reconstruyendo la tabla original desde *v_control_lote_almacen_compat*. En *db/tp_desnormalizacion_top_categorias.sql* (100.000 pedidos / 300.000 líneas), la consulta normalizada demoró **57,108 ms** (Parallel Seq Scan) frente a **0,071 ms** (~800× más rápido) sobre *venta_categoria_dia*, con **0 filas desincronizadas** en la auditoría *FULL OUTER JOIN*.

• **Unidad 5 (Simulacro de acceso indebido y anonimización):** Se reconstruyó a partir de *docs/log_simulacro_anonimizado.txt* la secuencia de ataque: 3 intentos fallidos sobre *fn_autenticar* en 4 segundos, lectura masiva (*SELECT * FROM cliente LIMIT 500*) e intento fallido de escalamiento (*GRANT admin_datos TO rol_app_lectura* bloqueado con *ERROR: permission denied*). **Propuesta de contención:** rotación de credenciales, bloqueo IP perimetral e invalidación de sesiones. En *sql/usuario_anon.sql* se verificó equivalencia exacta sobre **20.000 clientes** anonimizados.

• **Unidad 6 (Simulacro de desastre contra reloj e informe post-incidente):** Ejecutado con *db/restore_test.ps1* desde el respaldo custom *food_store_20261101_125256.dump* hacia la instancia aislada *foodstore_restore_test*. **RTO alcanzado: 3,52 segundos**; **RPO observado: ~24 minutos**. Conteos verificados sin pérdida: 9 categorías, 20.000 clientes, 10.000 productos, 100.000 pedidos y 300.000 detalles, con **0 violaciones CHECK**. Como mejora post-incidente se hizo idempotente la limpieza previa de la base de prueba.

• **Unidad 7 (Comparación MongoDB vs. PostgreSQL sobre pedidos reales IDs 1 a 5):** Se verificó coincidencia exacta de cantidades e IDs entre ambos motores: todos los pedidos migrados = 5 (IDs 1–5); estado *CONFIRMADO* = 3 (IDs 1, 2, 4); pedidos con *producto_id = 80* = 1 (ID 1); pedidos con *total_pedido > 2000* usando *NumberDecimal* = 4 (IDs 1, 2, 3, 5). Se seleccionó **PostgreSQL como fuente de verdad transaccional ACID** y **MongoDB como proyección documental de lectura** para pedidos cerrados completos.

3. Unidad 8 · Base Móvil SQLite, Sincronización y Casos C1–C7

La tablet opera sin conexión durante la feria consultando un catálogo local (*producto_cache*) y registrando borradores atómicos (*pedido_borrador*, *item_borrador*, *cola_sync*) definidos como tablas **STRICT** en *movil/schema_sqlite.sql* con *PRAGMA foreign_keys = ON* y precios en centavos enteros (*precio_centavos*). Un resultado **SIMULADO_APROBADO** valida la coincidencia contra la instantánea exportada por *tp8_servidor.v_snapshot_movil*, sin confirmar ventas ni descontar stock en PostgreSQL.

Caso	Preparación y Operación Probada	Resultado Verificado (25/25 tests OK)
C1	Consultar catálogo cacheado sin conexión a PostgreSQL.	Catálogo local disponible (productos activos IDs 10 y 15).
C2	Crear borrador, subir precio en servidor (450000 → 460000 centavos), exportar y evaluar. Luego aceptar nuevo precio y reenviar con UUID nuevo.	Primero **CONFLICTO**; tras aceptar precio y emitir nuevo UUID resulta **SIMULADO_APROBADO** conservando la traza previa.
C3	Solicitar más unidades (2) que el stock disponible en la instantánea (1).	**RECHAZADO** por stock insuficiente.
C4	Dar de baja lógica el producto 10 después del borrador, exportar CSV e importar cache.	**RECHAZADO**; desaparece del catálogo visible pero permanece su fila con *eliminado=1* (lápida) sin romper FKs.
C5	Enviar borrador sin modificaciones en el servidor.	**SIMULADO_APROBADO**; PostgreSQL permanece invariante (pedidos=1, stock 10=10, stock 15=5).
C6	Repetir el mismo *solicitud_uuid* y reejecutar el semáforo.	Error de PK (UNIQUE); cantidad de solicitudes y estado final previo conservados (idempotencia).
C7	Insertar cantidad cero (CHECK) y producto inexistente 999 (FK).	Errores CHECK y FK capturados; **ROLLBACK** atómico sin dejar filas huérfanas.

Arquitectura de Integración de los Tres Motores y Mantenimiento

Motor	Datos Almacenados	Autoridad y Plan de Mantenimiento
PostgreSQL 18	Productos, precios, stock, clientes, pedidos, detalles, roles y auditoría.	**Fuente de verdad operativa (ACID).** Respaldo diario *pg_dump -Fc* y simulacro semanal *restore_test.ps1* (RTO 3,52 s).
MongoDB (U7)	Colección *foodstore_nosql.pedidos* con cliente embebido, arreglo *lineas* y *NumberDecimal*.	**Proyección documental de lectura.** Respaldo con *mongodump* o reconstrucción desde PostgreSQL verificando IDs.
SQLite 3.37+ (U8)	*producto_cache*, *pedido_borrador*, *item_borrador* y *cola_sync*.	**Autoridad sobre el borrador local offline.** Respaldo consistente de borradores y verificación con *PRAGMA integrity_check* y *foreign_key_check*.

4. Code Review Final Asistido por IA y Decisiones Humanas (U4 a U8)

Se realizó la revisión asistida por IA de todos los scripts SQL, NoSQL, PowerShell y Python de la cursada, contrastando cada sugerencia mediante pruebas empíricas aisladas (*movil/pruebas_revision_postgresql.sql* y *docs/evidencia_revision.md*) y documentando la bitácora completa en *BITACORA.md* y *docs/DUIA.md*.

Hallazgo / Propuesta de la IA	Decisión Humana	Fundamento Técnico y Comprobación Empírica
U4 · Columna *subtotal* en *detalle_pedido* La IA sugirió eliminar *subtotal* alegando riesgo de desincronización frente a *cantidad * precio_unitario*.	**DESCARTADO**	En *db/preparar_food_store.sql* la columna está definida como *GENERATED ALWAYS AS (cantidad * precio_unitario) STORED*. Se comprobó en PostgreSQL 18 que insertar 2×4500 calcula 9000, actualizar cantidad a 3 recalcula 13500 y cualquier intento de sobrescribir *subtotal* es rechazado por el motor con **SQLSTATE 428C9**.
U4 · Relación entre 3FN y FNBC La IA detectó que un comentario afirmaba que violar FNBC en *control_lote_almacen* implicaba violar 3FN.	**ACEPTADO Y CORREGIDO**	Las claves candidatas son *{lote_id, deposito_id}* y *{lote_id, responsable_control_id}*; como los 3 atributos son primos, la tabla cumple 3FN aunque viole FNBC por *FD2*. Se corrigió en *db/tp_fnbc_control_lote.sql*.
U5 · Auditoría de *sql/roles.sql* La IA sugirió simplificar permisos con *GRANT ALL PRIVILEGES ON DATABASE food_store TO app_web;*.	**DESCARTADO**	Rechazado por violar el **principio de mínimo privilegio**. Se mantuvo la separación entre roles de grupo NOLOGIN y roles de login, restringiendo a *app_web* únicamente a *SELECT, INSERT, UPDATE* sobre tablas operativas.
U5 · Función *fn_autenticar* El code review advirtió que *fn_autenticar* verifica la existencia del correo pero ignora el argumento contraseña.	**ACEPTADO (Documentado)**	Comprobado en esquema temporal *pg_temp*. Se documenta que *fn_autenticar* fue creada como stub de laboratorio para generar trazas en el simulacro de auditoría de U5 y que en producción debe validar hashes con *pgcrypto*.
U7 · Eficiencia de cruces en MongoDB La IA afirmó inicialmente que cruzar colecciones en MongoDB era más rápido que *JOIN* en PostgreSQL.	**DESCARTADO / REFUTADO**	Contrastado con la documentación oficial de MongoDB: *$lookup* es costoso y el modelo documental solo aventaja cuando se recupera el documento embebido completo (pedido + cliente + líneas).
U8 · Importación de booleanos CSV (*t/f*) Tratar los booleanos exportados por PostgreSQL como enteros directos en tablas *STRICT* de SQLite.	**ACEPTADO Y CORREGIDO**	Se implementó la tabla de staging *snapshot_servidor* que recibe *'t'/'f'* como texto y los convierte explícitamente a *1/0* en *movil/cargar_cache.sql* conservando lápidas (*eliminado=1*).

5. Trazabilidad de Specs (Kiro), Bitácora y Defensa

• **Specs de Kiro completas:** *.kiro/specs/fnbc-y-desnormalizacion/* (U4), *.kiro/specs/roles-food-store/* (U5), *.kiro/specs/mantenimiento-u6/* (U6) y *.kiro/specs/movil-sqlite-sync/* junto con *docs/kiro/* (U8).
• **Bitácora de IA y DUIA:** *BITACORA.md*, *docs/bitacora_ia.md* y *docs/DUIA.md* registran todos los prompts, herramientas (Kiro y OpenCode) y validaciones realizadas por el equipo.
• **Verificación reproducible:** *python -m unittest discover -s movil/tests -v* ejecuta los 25 tests automatizados de la capa móvil e integración en menos de 1 segundo.